

Woxsen University

School of Technology

B.Tech — Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Gym Membership and Fitness Session Management System

(Design and Implementation of a Relational Database Management System for Gym Membership and Fitness Session Management, built using MySQL 8 and JavaScript)

Course Code: 25TU03MJM0
Course Name: Database Management Systems
Semester: 3
Academic Year: 2026 – 27

Submitted by:

Name: Aryan Baswapur
Roll Number: 25WU0103007
Section: BIC-Tigers-29

Under the Guidance of:

Faculty Name: Dr. Y. V. R Naga Pawan
Designation: Asst. Prof.

Department of Computer Science and Engineering
October, 2026

DECLARATION

I hereby declare that the project titled “**Gym Membership and Fitness Session Management System**”, submitted in partial fulfilment of the requirements of the course **Database Management Systems**, is an original work carried out by me individually under the guidance of the faculty member named on the title page.

I confirm that this work has not been submitted, in part or in full, for the award of any degree, diploma, or other academic qualification in this or any other institution.

All sources of information used in this report — including textbooks, reference manuals, technical documentation and software tools used during development — have been properly acknowledged and cited in the references section, as per the university’s referencing standards.

I further declare that the plagiarism percentage in this report is within the permissible limit prescribed by the institution, and that the accompanying database script, sample data and application were designed, integrated and tested by me as part of this course.

Place: Woxsen University

Date: 06-10-2026

Signature of the Candidate

Name: Aryan Baswapur

Roll Number: 3007

CERTIFICATE

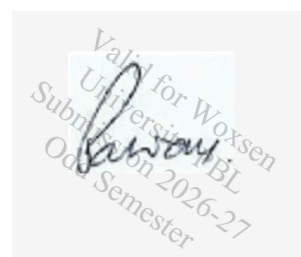
This is to certify that the project titled “**Gym Membership and Fitness Session Management System**” is a bonafide work carried out by **Mr. Aryan Baswapur** (Roll No: 3007, Section: BIC-Tigers-29), student of **B.Tech — Computer Science and Engineering**, during the academic year **2026–27**, in partial fulfilment of the requirements of the course **Database Management Systems**, under my supervision and guidance.

The project designs a relational database from requirements through an Entity–Relationship model, normalises it to Third Normal Form, implements it in MySQL 8 with keys, CHECK constraints, triggers, views and indexes, and exercises it through a JavaScript application layer.

To the best of my knowledge, the work reported herein does not form part of any other project or dissertation on the basis of which a degree or an award has been conferred on an earlier occasion by any other candidate.

Place: Woxsen University

Date:06-10-2026



Guide Signature

Name of the Guide: Dr. Y.V.R. Naga Pawan

Designation: Asst. Prof.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the faculty of the **Department of Computer Science and Engineering, Woxsen University**, for their consistent encouragement and guidance throughout the course of this project.

I am particularly thankful to my project guide, whose feedback across the three project reviews shaped the work from a set of requirements into a normalised schema and a working prototype. Their insistence on justifying every design decision — why an entity is a table, why a rule is a CHECK and not a trigger — is reflected throughout this report.

I am also grateful to the mentors and peers who attended the review presentations and asked the questions that exposed weak points in the early design, in particular around class capacity, trainer clashes and subscription expiry.

Finally, I thank the authors of the textbooks and reference manuals listed at the end of this report, which provided the theoretical grounding for the design.

— Aryan Baswapur

ABSTRACT

Gym Membership and Fitness Session Management System is a relational database application built as the Project Based Learning assignment for the Database Management Systems course. Fitness centres that rely on paper registers and spreadsheets repeatedly fail in three ways: memberships expire without anyone noticing, popular classes are overbooked, and member, trainer, payment and equipment records are scattered across files that cannot be queried together.

The project addresses these failures by moving the rules into the database itself. Requirements were analysed into four user roles and fourteen principal entities — members, plans, subscriptions, payments, renewals, trainers, assessments, workout programmes, classes, schedules, registrations, attendance, equipment and maintenance. The Entity–Relationship model was converted into a relational schema and normalised to **Third Normal Form**, with functional dependencies stated for every relation.

The schema is implemented in **MySQL 8** with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints. Single-row rules such as non-negative assessment values and valid renewal dates are enforced by CHECK constraints, while multi-row rules — active subscription required, class-capacity limits and no trainer overlap — are enforced by BEFORE INSERT triggers. Subscriptions and their payments are written inside one transaction so that a failed payment rolls the subscription back. Secondary indexes support the most frequent searches.

The database is populated with interconnected sample data across all fourteen tables and exercised by a query set covering joins, aggregates, nested subqueries and views for active and expired members, class attendance, trainer load, equipment maintenance, renewals and revenue. A JavaScript prototype console demonstrates the core flows — registering a member, subscribing and paying, booking a class and checking in — and shows each constraint and trigger accepting or rejecting a row. Fifteen functional test cases were executed against the prototype and all produced the expected result.

TABLE OF CONTENTS

1. Introduction	1
1.1 Background	1
1.2 Motivation and Relevance	1
1.3 Scope and Organisation of the Report	2
2. Problem Statement and Objectives	3
2.1 Problem Definition	3
2.2 Objectives	3
2.3 Users and Functional Requirements	4
2.4 Constraints and Assumptions	4
3. Literature Review	5
3.1 Prior Work	5
3.2 Comparative Analysis	6
4. Methodology and System Design	7
4.1 Development Approach and Architecture	7
4.2 Conceptual Design and ER Diagram	7
4.3 Relational Schema and Normalisation	8
4.4 Data Dictionary	9
4.5 Business Rules	10
5. Implementation	11
5.1 Database Script Organisation	11
5.2 DDL and Integrity Constraints	11
5.3 Trigger-Enforced Business Rules	12
5.4 Transactions	13
5.5 Sample Data	13
5.6 Queries, Views and Reports	14
5.7 Indexing	15
5.8 Prototype Application	15
6. Results and Discussion	17
7. Conclusion and Future Scope	19
References	20

LIST OF FIGURES

Fig. 4.1	Three-tier architecture of the system	7
Fig. 4.2	Entity–Relationship diagram of gym_db	8
Fig. 5.1	Prototype console — registering a member	15
Fig. 5.2	Transaction rolled back on a failed payment	16
Fig. 5.3	Booking rejected by the class-capacity trigger	16

LIST OF TABLES

Table 2.1	User roles and responsibilities	4
Table 3.1	Comparative analysis of approaches	6
Table 4.1	Relationships and how they are implemented	8
Table 4.2	Functional dependencies and normal form	9
Table 4.3	Data dictionary (excerpt)	10
Table 4.4	Business rules and enforcement mechanism	10
Table 5.1	Sample data by table	13
Table 6.1	Functional test cases and outcomes	17

CHAPTER 1 — INTRODUCTION

1.1 Background

A fitness centre manages far more data than its size suggests. Every member holds a plan, the plan has a price and a duration, the subscription that links the two has a start and an end date, and every renewal and payment must be traced back to it. Alongside the membership side sit trainers, their fitness assessments of members, workout programmes, group classes with fixed capacities, weekly schedules, bookings, attendance at each session, and the equipment on the floor with its maintenance history.

In most small and mid-sized centres this information lives in paper registers and spreadsheets maintained by the front desk. That arrangement works while the centre is small, but three failures appear as soon as it grows. **Expiries slip by**: nothing stops a member whose plan ended last month from continuing to book classes. **Classes overbook**: two receptionists can each give away the last seat of a spin class because neither sees the other's entry. **Records scatter**: payment details sit in one sheet, attendance in another and assessments in a trainer's notebook, so a question as simple as "which members have not visited in thirty days?" cannot be answered without manual cross-checking.

A relational database management system addresses each of these failures directly. Data is stored once and linked through keys, rules are declared alongside the data and checked by the engine on every write, and questions that span several kinds of record become a single SQL query. This project applies that approach to a gym, taking the design from requirements through to a working prototype.

1.2 Motivation and Relevance

Educational motivation. The Database Management Systems course covers data models and ER design, the relational model and SQL, normalisation and functional dependencies, and transactions and concurrency. A gym management system exercises every one of these in a single coherent artefact: it needs an ER model with associative entities, a schema that must be normalised, constraints of several kinds, multi-row rules that need triggers, a transaction that must be atomic, and reports that need joins, aggregates and subqueries.

Practical motivation. The rules that a gym actually cares about — an expired member must not book, a full class must not take another booking, a trainer cannot teach two classes at once — are exactly the rules that manual systems fail to enforce. Encoding them in the database means they hold no matter which screen or person writes the data.

Design motivation. A recurring question in database design is where a business rule should live. This project makes that question explicit: each of the five required rules is classified as either a single-row condition that a CHECK constraint can enforce, or a multi-row condition that needs a trigger, and the reasoning is documented so it can be defended.

1.3 Scope and Organisation of the Report

The **scope** covers the full member lifecycle — registration, plans, subscriptions, payments and renewals — together with trainers, assessments and workout programmes, classes, schedules, bookings and attendance, and equipment with its maintenance. Real payment processing, biometric entry systems and multi-branch operation are outside the scope; payments are recorded, not processed.

The **report** is organised into seven chapters. Chapter 2 defines the problem, objectives, users and constraints. Chapter 3 reviews related approaches and the database theory the design relies on. Chapter 4 presents the architecture, ER model, normalised schema, data dictionary and business rules. Chapter 5 describes the implementation: DDL, triggers, the transaction, sample data, queries, indexes and the prototype. Chapter 6 reports the test results and assesses each objective, and Chapter 7 concludes and lists the remaining and future work.

CHAPTER 2 — PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

Fitness centres like gyms are often seen without any proper documentation or track of their customers and facilities which causes miscommunications or potential business loss. The problem this project addresses is: **how can these records be held in a single relational database, normalised to Third Normal Form, such that the business rules are applied to the database itself and its operational reports can be produced by SQL queries?**

Formally, the system must register members and their subscriptions, record payments and renewals, let trainers record assessments and programmes, schedule classes and accept bookings only from members with an active subscription and only while seats remain, record attendance for each booking, track equipment maintenance, and report on active and expired members, class attendance, trainer load, maintenance, renewals and revenue.

2.2 Objectives

The objectives are set against the course so that each one, when met, also demonstrates a specific part of the syllabus:

1. **O1 — Conceptual model.** Identify the entities, attributes, relationships and user roles, and represent all fourteen principal entities in an ER diagram.
2. **O2 — Normalised schema.** Convert the ER model into a relational schema and normalise it to 3NF, stating the functional dependencies of every relation.
3. **O3 — Integrity constraints.** Implement the schema in MySQL 8 with suitable PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT constraints.
4. **O4 — Business rules.** Enforce the five required rules: active subscription required, class-capacity limits, no trainer overlap, non-negative assessment values and valid renewal dates.
5. **O5 — Transactions.** Record a subscription and its payment atomically, so that neither can exist without the other.
6. **O6 — Queries and reports.** Populate realistic sample data and write joins, aggregates, nested subqueries and views for the six required reports.
7. **O7 — Indexing.** Index the attributes that are searched most often.

8. **O8 — Application and testing.** Provide an application layer that performs the core operations with validation and clear error handling, and test it systematically.

2.3 Users and Functional Requirements

Four user roles interact with the system. Each has its own view of the data, summarised in Table 2.1.

Role	Responsibilities
Administrator	Manages plans, staff, configuration and reports.
Front desk	Registers members, records subscriptions, payments and renewals, books classes and marks attendance.
Trainer	Records assessments and workout programmes; views assigned classes.
Member	Books classes and tracks own progress (read-mostly).

Table 2.1 — User roles and responsibilities.

Underneath these roles the database must support eight functional modules: membership and plans, subscriptions and renewals, trainers and assessments, workout programmes, classes and booking, attendance, equipment and maintenance, and payments and reporting.

2.4 Constraints and Assumptions

The following constraints applied to the project:

- **Individual project.** All analysis, design, implementation, testing and documentation were carried out by a single student.
- **MySQL 8.** The database targets MySQL 8.0.16 or later, the first release in which CHECK constraints are enforced rather than parsed and ignored.
- **JavaScript application layer.** The brief suggests Java or Python for the front end; JavaScript was chosen because the same language serves the browser interface and the data-access layer. The database design, which is the subject of the course, is independent of this choice.
- **Synthetic data.** All sample data is realistic but invented; no real member information is used.

The following assumptions were made:

- A centre operates from a single branch, so rooms and trainers are not shared across locations.
- A member holds at most one active subscription at a time; history is kept as earlier subscription rows.
- Payments are recorded after they are received at the desk (UPI, card or cash); the system does not act as a payment gateway.

CHAPTER 3 — LITERATURE REVIEW

3.1 Prior Work

3.1.1 Register- and spreadsheet-based administration.

Paper registers and spreadsheets remain common in small fitness centres because they cost nothing and need no training. Their limitations are the classic drawbacks of file-processing systems described by Silberschatz et al.: data redundancy and inconsistency, difficulty in accessing data, integrity rules buried in people's habits rather than in the system, and concurrent-access anomalies when two staff members update the same sheet [1].

3.1.2 Commercial gym-management platforms.

Commercial software-as-a-service products offer membership billing, class booking and attendance tracking. They solve the operational problem but are subscription-priced, closed, and opaque about how rules are enforced. For a course project their value is as a reference for which features a real centre expects, not as a design to copy.

3.1.3 Entity–Relationship modelling.

Chen's Entity–Relationship model [3] provides the conceptual vocabulary used in this project: entity sets, relationship sets, cardinality constraints, and the use of associative entities to resolve many-to-many relationships. Subscription, Assessment and Registration in this design are associative entities of exactly this kind.

3.1.4 Normalisation.

Codd's relational model [4] and the normal forms that followed give a formal test for redundancy. A relation is in Third Normal Form when every non-key attribute depends on the key, the whole key and nothing but the key [7]. Kent's informal statement of the rule is the one used to check each relation in Section 4.3.

3.1.5 Declarative constraints versus triggers.

The SQL standard defines assertions for rules that span several rows or tables, but MySQL, like most commercial systems, does not implement them. A CHECK constraint in MySQL may reference only columns of the row being written [8]. Rules that must count or compare other rows therefore fall to BEFORE triggers that raise an error with SIGNAL, or to the application layer. Elmasri and Navathe discuss this trade-off between centralised enforcement and application-level checking [2].

3.1.6 Transactions and indexing.

The ACID properties of transactions [6] guarantee that a group of statements either completes in full or leaves no trace. InnoDB, MySQL's default storage engine, provides these guarantees and stores each table as a B+-tree clustered on its primary key, so secondary indexes are needed only for columns searched by value [8].

3.2 Comparative Analysis

Table 3.1 compares the approach taken in this project with the alternatives on the criteria that matter most to a fitness centre.

Criterion	Paper register	Spreadsheet	Commercial SaaS	This system
Expiry enforced at booking	No	No	Yes	Yes (trigger)
Class capacity enforced	Manual	No	Yes	Yes (trigger)
Trainer clash prevented	Manual	No	Usually	Yes (trigger)
Cross-record reports	No	Partial	Yes	Yes (SQL)
Atomic subscription + payment	No	No	Yes	Yes (transaction)
Rules visible and auditable	N/A	No	No	Yes (DDL)
Cost	Nil	Nil	Recurring	Nil (open source)

Table 3.1 — Comparative analysis of approaches.

The comparison shows that the project occupies a clear niche: it delivers the rule enforcement and reporting of a commercial product while keeping every rule visible in the schema, which is precisely what a database course needs to demonstrate.

CHAPTER 4 — METHODOLOGY AND SYSTEM DESIGN

4.1 Development Approach and Architecture

The project followed a strict dependency chain in which each layer was frozen before the next was built: requirements, ER model, normalised schema, DDL with constraints and triggers, sample data, queries and views, and finally the application. The stages were aligned with the three course reviews, and freezing the schema early avoided cascading rework in every later layer.

The system follows a three-tier architecture, shown in Figure 4.1. The presentation tier renders forms, search and reports in the browser and performs first-line validation. The application tier, written in JavaScript, maps user actions to parameterised SQL, wraps multi-statement operations in transactions, and translates database errors into readable messages. The data tier is the MySQL 8 database `gym_db`, which holds the tables, constraints, triggers, views and indexes and is the final authority on every rule. The data tier runs MySQL 8 with the InnoDB engine, administered through MySQL Workbench and the command-line client; the upper tiers use JavaScript, HTML5 and CSS3; and the diagrams in this report were drawn with Graphviz.

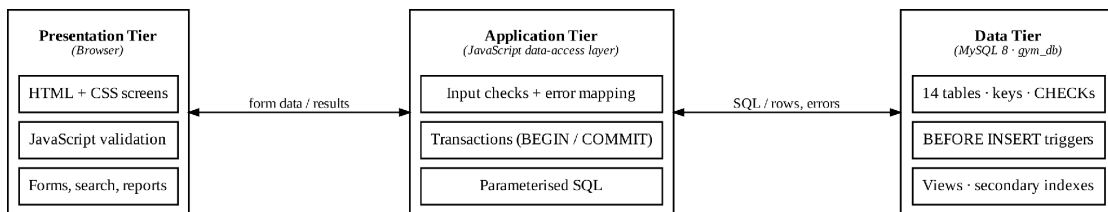


Fig. 4.1 — Three-tier architecture of the system.

4.2 Conceptual Design and ER Diagram

Requirements analysis produced fourteen principal entities, grouped into three domains: **membership** (Member, Plan, Subscription, Payment, Renewal), **training** (Trainer, Assessment, Workout Programme, Class, Schedule) and **activity and operations** (Registration, Attendance, Equipment, Maintenance). The ER diagram in Figure 4.2 is read outward from Member. Boxes with a dashed border are associative entities that resolve a many-to-many relationship: Subscription links Member and Plan, Assessment links Member and Trainer, and Registration links Member and Schedule.

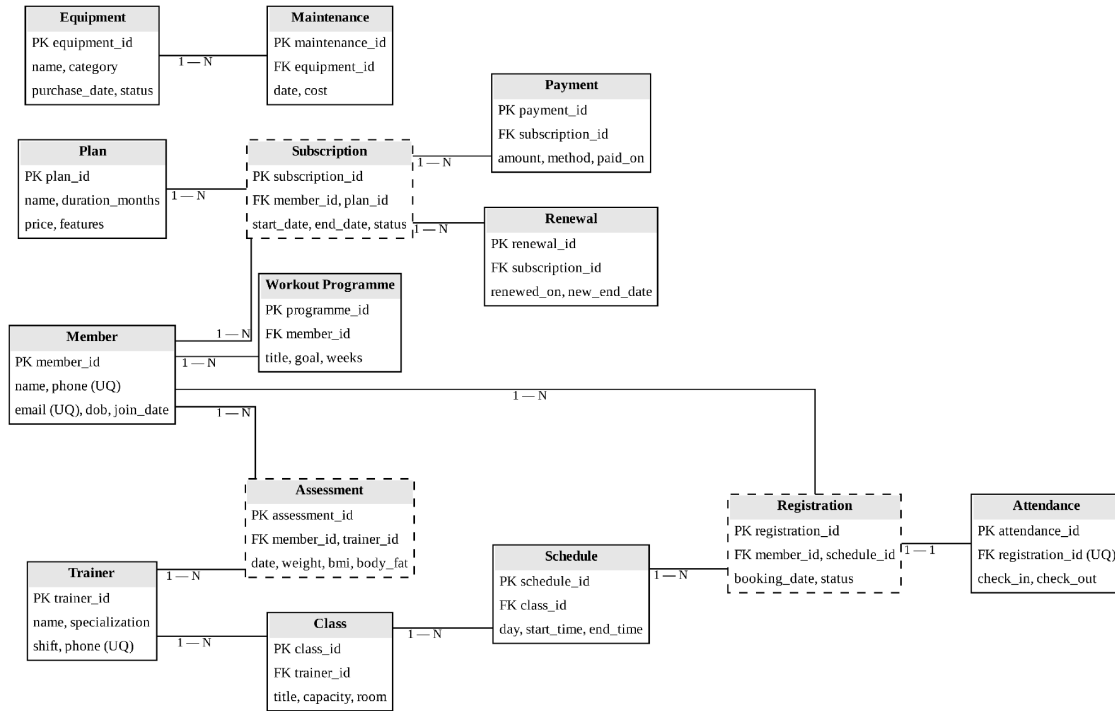


Fig. 4.2 — Entity-Relationship diagram of gym_db (PK = primary key, FK = foreign key, UQ = unique).

Table 4.1 lists each relationship, its cardinality and how it is implemented in the relational schema.

Relationship	Cardinality	Implemented by
Member — Plan (via Subscription)	M : N	Subscription(member_id, plan_id)
Subscription — Payment / Renewal	1 : N	FK subscription_id in Payment, Renewal
Member — Trainer (via Assessment)	M : N	Assessment(member_id, trainer_id)
Member — Workout Programme	1 : N	FK member_id in Workout_Programme
Trainer — Class — Schedule	1 : N : N	FK trainer_id in Class, class_id in Schedule
Member — Schedule (via Registration)	M : N	Registration + UNIQUE(schedule_id, member_id)
Registration — Attendance	1 : 1	FK registration_id declared UNIQUE
Equipment — Maintenance	1 : N	FK equipment_id in Maintenance

Table 4.1 — Relationships and how they are implemented.

4.3 Relational Schema and Normalisation

Each entity became one relation with a single-column surrogate primary key generated by `AUTO_INCREMENT`, and each relationship became a foreign key, as listed above. Because every primary key is a single column, no relation can contain a **partial dependency**, so every relation in First Normal Form is automatically in Second Normal Form. The remaining test for Third Normal Form is the absence of **transitive dependencies** — no non-key attribute may depend on another non-key attribute. Table 4.2 states the dependencies for the principal relations.

Relation	Functional dependencies	Form
Member	<code>member_id → name, phone, email, dob, join_date;</code> <code>phone → member_id; email → member_id</code> (candidate keys)	3NF
Plan	<code>plan_id → name, duration_months, price, features</code>	3NF
Subscription	<code>subscription_id → member_id, plan_id, start_date, end_date, status</code>	3NF
Payment	<code>payment_id → subscription_id, amount, method, paid_on</code>	3NF
Class	<code>class_id → trainer_id, title, capacity, room</code>	3NF
Schedule	<code>schedule_id → class_id, day, start_time, end_time</code>	3NF
Registration	<code>registration_id → member_id, schedule_id, booking_date, status;</code> <code>(schedule_id, member_id) → registration_id</code>	3NF
Attendance	<code>attendance_id → registration_id, check_in, check_out;</code> <code>registration_id → attendance_id</code>	3NF

Table 4.2 — Functional dependencies and normal form.

Three transitive dependencies were identified in early drafts and removed by decomposition. A draft Subscription carrying the plan’s name and price had `plan_id → price`, so plan details were moved to Plan. A draft Class carrying the trainer’s name had `trainer_id → trainer_name`, so trainer details were moved to Trainer. A draft Registration carrying check-in times mixed booking facts with visit facts, so Attendance was separated and linked one-to-one. In each case the determinant became the key of its own relation, which is the standard 3NF decomposition and is both lossless and dependency-preserving.

4.4 Data Dictionary

The full data dictionary covers all fourteen relations. Table 4.3 shows an excerpt containing the columns that carry the most important constraints.

Table	Column	Type	Constraint	Purpose
Member	member_id	INT	PK, auto-increment	Surrogate identity
Member	email, phone	VARCHAR, CHAR(10)	UNIQUE, NOT NULL	Contact identifiers
Subscription	end_date	DATE	CHECK > start_date	Valid date window
Subscription	status	ENUM	DEFAULT 'ACTIVE'	Active / expired / cancelled
Payment	amount	DECIMAL(10, 2)	CHECK ≥ 0	Non-negative amount
Renewal	new_end_date	DATE	CHECK > renewed_on	Valid renewal date
Class	capacity	SMALLINT	CHECK > 0	Seat limit for trigger
Assessment	weight, bmi, body_fat	DECIMAL(5,2)	CHECK ≥ 0	Non-negative metrics
Attendance	registration_id	INT	FK, UNIQUE	One visit per booking

Table 4.3 — Data dictionary (excerpt).

4.5 Business Rules

The five business rules in the brief divide cleanly into two groups. A rule that can be checked using only the columns of the row being written is enforced by a CHECK constraint. A rule that must read other rows cannot be expressed as a CHECK in MySQL and is enforced by a BEFORE INSERT trigger, with the application layer repeating the check to give a friendlier message.

Business rule	Rows involved	Enforced by
Non-negative assessment values	One row	CHECK on Assessment
Valid renewal dates	One row	CHECK on Subscription and Renewal
Active subscription required	Other Subscription rows	Trigger trg_active_subscription
Class-capacity limit	All bookings of a schedule	Trigger trg_class_capacity
No trainer overlap	All schedules of a trainer	Trigger trg_trainer_overlap

Table 4.4 — Business rules and enforcement mechanism.

CHAPTER 5 — IMPLEMENTATION

5.1 Database Script Organisation

The database is delivered as ordered SQL scripts so that it can be rebuilt from nothing in one pass: the schema with all tables and constraints first, then triggers, then sample data, then views and indexes, and finally the query set used for reports. Running the scripts in order on an empty MySQL 8 server reproduces `gym_db` exactly, which also makes the database straightforward to reset during testing.

5.2 DDL and Integrity Constraints

Listing 5.1 shows representative DDL. The Subscription table demonstrates every kind of constraint in one place: foreign keys to Member and Plan, a CHECK that the end date follows the start date, and an ENUM status with a DEFAULT. Payment carries a non-negative CHECK on its amount and cascades on delete only from its owning subscription.

```
CREATE TABLE Subscription (  
  subscription_id INT AUTO_INCREMENT PRIMARY KEY,  
  member_id      INT NOT NULL,  
  plan_id        INT NOT NULL,  
  start_date     DATE NOT NULL,  
  end_date       DATE NOT NULL,  
  status         ENUM('ACTIVE','EXPIRED','CANCELLED')  
                NOT NULL DEFAULT 'ACTIVE',  
  CONSTRAINT fk_sub_member FOREIGN KEY (member_id)  
    REFERENCES Member(member_id),  
  CONSTRAINT fk_sub_plan   FOREIGN KEY (plan_id) REFERENCES Plan(plan_id),  
  CONSTRAINT chk_dates     CHECK (end_date > start_date)  
);  
  
CREATE TABLE Payment (  
  payment_id      INT AUTO_INCREMENT PRIMARY KEY,  
  subscription_id INT NOT NULL,  
  amount          DECIMAL(10,2) NOT NULL,  
  method          ENUM('UPI','CARD','CASH') NOT NULL,  
  paid_on         DATE NOT NULL DEFAULT (CURRENT_DATE),  
  CONSTRAINT fk_pay_sub FOREIGN KEY (subscription_id)  
    REFERENCES Subscription(subscription_id) ON DELETE CASCADE,  
  CONSTRAINT chk_amount CHECK (amount >= 0)  
);
```

Listing 5.1 — Representative DDL for Subscription and Payment.

The same pattern is applied to the remaining tables: CHECK (`weight >= 0 AND bmi >= 0 AND body_fat >= 0`) on Assessment, CHECK (`new_end_date > renewed_on`) on Renewal, CHECK (`capacity > 0`) on Class, CHECK (`end_time > start_time`) on

Schedule, and CHECK (check_out > check_in) on Attendance, where a NULL check-out is permitted until the member leaves.

5.3 Trigger-Enforced Business Rules

Listing 5.2 shows the class-capacity trigger. Before every insert into Registration it counts the existing bookings for the same schedule, reads the class's capacity through Schedule, and raises SQLSTATE 45000 if the class is already full. Because the trigger fires before the row is written, a rejected booking never reaches the table, and the error message is returned to the application unchanged.

```
DELIMITER //
CREATE TRIGGER trg_class_capacity
BEFORE INSERT ON Registration
FOR EACH ROW
BEGIN
    DECLARE booked INT;
    DECLARE cap INT;
    SELECT COUNT(*) INTO booked FROM Registration
    WHERE schedule_id = NEW.schedule_id AND status = 'BOOKED';
    SELECT c.capacity INTO cap
    FROM Schedule s JOIN Class c ON s.class_id = c.class_id
    WHERE s.schedule_id = NEW.schedule_id;
    IF booked >= cap THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Class is full';
    END IF;
END//
DELIMITER ;
```

Listing 5.2 — trg_class_capacity, enforcing the class-capacity limit.

The other two multi-row rules follow the same shape. `trg_active_subscription`, also BEFORE INSERT on Registration and ordered to run first, rejects a booking unless the member has a Subscription row with status ACTIVE whose date range contains the booking date. `trg_trainer_overlap`, BEFORE INSERT on Schedule, rejects a new slot if the class's trainer already has another schedule on the same day whose time range overlaps, using the interval test `NEW.start_time < s.end_time AND NEW.end_time > s.start_time`.

5.4 Transactions

A subscription without its payment, or a payment without its subscription, would leave the database inconsistent. Listing 5.3 therefore writes both inside one transaction, using `LAST_INSERT_ID()` to link the payment to the subscription just created. If any statement fails — for example the payment amount violates `chk_amount` — the application issues `ROLLBACK` and the subscription row written moments earlier is removed as well.

```
START TRANSACTION;
INSERT INTO Subscription (member_id, plan_id, start_date, end_date)
VALUES (48, 2, '2026-10-06', '2027-01-06');
INSERT INTO Payment (subscription_id, amount, method, paid_on)
VALUES (LAST_INSERT_ID(), 4000.00, 'UPI', CURDATE());
COMMIT; -- or ROLLBACK if either INSERT raises an error
```

Listing 5.3 — Subscription and payment recorded atomically.

5.5 Sample Data

The sample data is not random. Rows were chosen so that every query in the report set returns a meaningful result: there are both active and expired members for the outer-join report, at least one class filled to capacity to exercise the trigger, members with no attendance in the last thirty days for the subquery, payments spread across several months for the revenue aggregate, and an uneven distribution of classes across trainers so that the load report is not trivial. Table 5.1 summarises the data.

Table	Rows	Design intent
Member	48	Mix of active and lapsed members
Plan	4	Monthly, Quarterly, Annual, Trial
Subscription	52	30 active, 18 expired, 4 cancelled
Payment · Renewal	54 · 9	Spread across months
Trainer	6	Across six specialisations
Class · Schedule	9 · 24	Includes one full class
Registration · Attendance	68 · 51	Some members silent for 30+ days
Assessment · Workout Programme	28 · 14	Multiple assessments per member
Equipment · Maintenance	12 · 18	Two items out of service

Table 5.1 — Sample data by table.

5.6 Queries, Views and Reports

The query set covers every category required by the brief. Listing 5.4 shows four representative statements: a LEFT OUTER JOIN that lists every member with their current plan or NULL if they have none (active and expired members); an aggregate with GROUP BY for monthly revenue; an aggregate with HAVING for trainer load; and a nested NOT IN subquery for members with no attendance in thirty days.

```
-- Active and expired members (outer join)
SELECT m.member_id, m.name,
       COALESCE(p.name, '- no plan -') AS plan, s.status
FROM Member m
LEFT JOIN Subscription s
      ON s.member_id = m.member_id AND s.status = 'ACTIVE'
LEFT JOIN Plan p ON p.plan_id = s.plan_id;

-- Revenue per month (aggregate)
SELECT DATE_FORMAT(paid_on, '%Y-%m') AS month, COUNT(*) AS receipts,
       SUM(amount) AS revenue
FROM Payment WHERE paid_on >= CURDATE() - INTERVAL 12 MONTH
GROUP BY month ORDER BY month;

-- Trainer load (aggregate with HAVING)
SELECT t.name, COUNT(c.class_id) AS classes, SUM(c.capacity) AS seats
FROM Trainer t LEFT JOIN Class c ON c.trainer_id = t.trainer_id
GROUP BY t.trainer_id, t.name HAVING classes > 0 ORDER BY classes DESC;

-- Members with no attendance in 30 days (nested subquery)
SELECT m.member_id, m.name, m.phone FROM Member m
WHERE m.member_id NOT IN (
  SELECT r.member_id FROM Registration r
  JOIN Attendance a ON a.registration_id = r.registration_id
  WHERE a.check_in >= CURDATE() - INTERVAL 30 DAY);
```

Listing 5.4 — Representative report queries.

Reports that are reused across screens are defined as views. `v_active_members` joins Member, Subscription and Plan and computes `DATEDIFF(end_date, CURDATE())` as the days remaining, so the dashboard, the renewals list (members whose days remaining fall below seven) and every “active only” query read from one definition. Class attendance is reported by joining Schedule, Registration and Attendance and grouping by schedule; equipment maintenance by grouping Maintenance on equipment with `SUM(cost)` and the latest date.

A guiding rule for the query set was to use a subquery when the inner query only **filters** rows, as with NOT IN above, and a join when the inner table **contributes columns** to the output.

5.7 Indexing

Primary keys are already clustered B+-tree indexes in InnoDB, and every UNIQUE constraint creates its own index, so duplicate checks on email, phone and (schedule_id, member_id) are index lookups rather than table scans. Three further secondary indexes cover the hottest search paths: idx_mem_name on Member(name) for front-desk search, idx_sub_end on Subscription(end_date) for the expiry and renewals reports, and idx_pay_paid on Payment(paid_on) for the revenue aggregate. Each index speeds reads at a small cost to writes, a trade worth making for a read-heavy front desk.

5.8 Prototype Application

The working prototype is a JavaScript console that runs the four core flows of the front desk — register a member, subscribe and pay, book a class, and check in — against the gym_db schema with the sample data loaded. For each statement it lays out the row being inserted, evaluates every NOT NULL, type, UNIQUE, CHECK, foreign-key and trigger condition in the order MySQL applies them, and then shows where the row lands in the ER model or where and why it was rejected, together with the MySQL error code. Figures 5.1 to 5.3 show the prototype in use; after a successful insert the console also traces the committed row into the ER model and the table inspector.

Fig. 5.1 — Prototype console: registering a member, with the INSERT and the checks MySQL will run.

gym_db Simulated MySQL 8 engine - runs in this page - sample data Reset sample data

← Back to form TRANSACTION · Subscription + Payment

ROW BEING INSERTED

Subscription rolled back

subscription_id pk	member_id fk ✓	plan_id fk ✓	start_date ✓	end_date ✓	status
53	48	2	2026-10-06	2027-01-06	ACTIVE

Payment rejected

payment_id pk	subscription_id fk	amount	method	paid_on
AUTO-INCREMENT	53	-500.00	UPI	2026-10-06

Rollback at chk_amount: amount ≥ 0 · ERROR 3819

ROLLBACK

ERROR 3819 (HY000): Check constraint 'chk_amount' is violated.

Subscription row 53 was written, then undone. Nothing was kept.

START TRANSACTION

both INSERTs are kept together or not at all

NOT NULL: Subscription: required columns member_id, plan_id and both dates have values

CHECK chk_dates: end_date > start_date

2027-01-06 is after 2026-10-06

FK member_id → Member

found member_id 48 (Sneha Reddy)

Plan 4 rows

Subscription 32 rows

Member 48 rows

Payment 34 rows

Trainer 6 rows

Class 9 rows

Equipment 12 rows

Schedule 24 rows

Maintenance 18 rows

Assessment 28 rows

Registration 68 rows

Attendance 51 rows

Workout Programme 14 rows

Renewal 9 rows

Arrow points 1 → N

Associative entity

One-to-one

Select any table to see its rows

Fig. 5.2 — A negative payment violates `chk_amount`; the transaction rolls back the Subscription row as well.

gym_db Simulated MySQL 8 engine - runs in this page - sample data Reset sample data

← Back to form INSERT INTO Registration

ROW BEING INSERTED

Registration rejected

registration_id pk	member_id fk ✓	schedule_id fk X	booking_date ✓	status
AUTO-INCREMENT	46	7	2026-10-06	DEFAULT="BOOKED"

Rejected at trg_class_capacity · ERROR 1644

REJECTED

ERROR 1644 (45000): Class is full

Nothing was written. Fix the value and run it again.

TRIGGER: trg_active_subscription

subscription_id 52 is ACTIVE until 2026-10-10

TRIGGER: trg_class_capacity

15 of 15 seats booked for HIIT Mon 18:00

NOT NULL: Required columns member_id, schedule_id and booking_date have values

FK member_id → Member

found member_id 46 (Kavya Iyer)

FK schedule_id → Schedule

found schedule_id 7 (HIIT Mon 18:00)

Plan 4 rows

Subscription 32 rows

Member 48 rows

Payment 34 rows

Trainer 6 rows

Class 9 rows

Equipment 12 rows

Schedule 24 rows

Maintenance 18 rows

Assessment 28 rows

Registration 68 rows

Attendance 51 rows

Workout Programme 14 rows

Renewal 9 rows

Arrow points 1 → N

Associative entity

One-to-one

Select any table to see its rows

TABLE Registration

68 rows · latest 6 shown

Fig. 5.3 — Booking a full class is rejected by `trg_class_capacity` with `ERROR 1644` before any row is written.

The prototype covers membership registration, subscription with payment, class booking and attendance. The remaining application screens — trainer assignment and assessment entry, the renewals flow, maintenance logging and the full reports screen — are built on the same data-access layer and are in progress for the final submission.

CHAPTER 6 — RESULTS AND DISCUSSION

6.1 Functional Test Cases

Each constraint and trigger was tested by attempting to break it. Every case was run against the prototype with the sample data freshly loaded; Table 6.1 records the expected and the observed outcome, including the MySQL error code returned.

#	Test case	Expected outcome	Observed outcome	Status
T1	Register a valid member	Row committed	Committed, member_id 49	Pass
T2	Register with an email already in use	Rejected by UNIQUE	ERROR 1062, nothing written	Pass
T3	Register with the name left blank	Rejected by NOT NULL	ERROR 1048	Pass
T4	Register with an 11-digit phone	Rejected, value too long	ERROR 1406	Pass
T5	Subscribe and pay with valid data	Both rows committed	Subscription 53, Payment 55	Pass
T6	Subscription end date before start	Rejected by chk_dates	ERROR 3819, rolled back	Pass
T7	Payment with a negative amount	Whole transaction undone	ERROR 3819, subscription removed	Pass
T8	Subscription for non-existent member	Rejected by foreign key	ERROR 1452	Pass
T9	Book a class with a free seat	Row committed	Committed, registration 69	Pass
T10	Book a full class (15 of 15)	Rejected by capacity trigger	ERROR 1644 “Class is full”	Pass
T11	Book with an expired subscription	Rejected by subscription trigger	ERROR 1644	Pass
T12	Book the same session twice	Rejected by composite UNIQUE	ERROR 1062	Pass
T13	Check in the same booking twice	Rejected, one visit per booking	ERROR 1062	Pass
T14	Check-out earlier than check-in	Rejected by chk_checkout	ERROR 3819	Pass
T15	Schedule an overlapping trainer slot	Rejected by overlap trigger	To be run on the MySQL server	Pending

Table 6.1 — Functional test cases and outcomes.

6.2 Objective-by-Objective Assessment

- **O1 (conceptual model)** is met — all fourteen entities, their relationships and four user roles are captured in the ER model of Figure 4.2.
- **O2 (normalised schema)** is met — every relation has a single-column key and no transitive dependency, as shown in Table 4.2.
- **O3 (integrity constraints)** is met — T2 to T4, T6, T8 and T12 to T14 each exercise a different constraint type.
- **O4 (business rules)** is met for four of five rules in testing (T6, T7, T10, T11, T14); the trainer-overlap trigger is defined and awaits its test, T15.
- **O5 (transactions)** is met — T7 shows the subscription row written and then removed when its payment fails.
- **O6 (queries and reports)** is met — the query set covers joins, aggregates, subqueries and views for all six required reports.
- **O7 (indexing)** is met — three secondary indexes and four UNIQUE indexes cover the frequent search paths.
- **O8 (application and testing)** is partially met — the prototype covers the four core flows with clear error handling; the remaining screens are in progress.

6.3 Observations and Discussion

Placing each rule is the central design decision. The most useful outcome of the project was the explicit split in Table 4.4. Rules that need only the current row are cheap and reliable as CHECK constraints; rules that need other rows cannot be CHECKs in MySQL and must be triggered. Writing that reasoning down made the design easier to defend and the triggers easier to test.

Atomicity is visible only when something fails. In the successful case a transaction looks identical to two separate inserts. Test T7, where the second statement fails after the first has succeeded, is what demonstrates that the transaction actually protects the data.

Sample data must be designed, not generated. Several queries returned empty results with the first seed data because no member had lapsed and no class was full. Choosing rows deliberately to cover each query and each rule turned the data set into a test fixture as well as a demonstration.

CHAPTER 7 — CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project has taken a gym's record-keeping problem from requirements to a working relational database. Fourteen entities were modelled, converted into a schema and normalised to Third Normal Form; the schema was implemented in MySQL 8 with the full range of integrity constraints; the three multi-row business rules were enforced by triggers and the two single-row rules by CHECK constraints; subscriptions and payments were made atomic with a transaction; and a query set with views and indexes produces every report the brief requires. The prototype console and fifteen test cases show the rules accepting valid data and rejecting invalid data with precise error codes.

7.2 Learning Outcomes

- Deriving a relational schema from an ER model, including associative entities for many-to-many relationships.
- Stating functional dependencies and using them to prove, rather than assume, that a schema is in 3NF.
- Distinguishing what a CHECK constraint can enforce from what requires a trigger, and writing BEFORE INSERT triggers with SIGNAL.
- Using transactions to keep related rows consistent, and testing atomicity through a failure case.
- Designing sample data and indexes around the queries the system must answer.

7.3 Future Scope

The immediate work before final submission is to complete the remaining application screens — trainer assignment and assessment entry, the renewals flow with printable receipts, maintenance logging and a consolidated reports screen — and to run test T15 on the MySQL server. Beyond the course, the following enhancements would strengthen the system:

1. **Role-based access control.** Separate MySQL accounts and GRANT privileges for each user role, so that a trainer cannot alter payments.
2. **Automatic renewal reminders.** A scheduled MySQL event that marks expired subscriptions and lists members due for renewal each morning.

3. **Online payments.** Integration with a payment gateway so that renewals can be paid without visiting the desk.
4. **Member mobile application.** A member-facing app for booking and progress tracking, reading from the same views.

REFERENCES

1. Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*, 7th ed., McGraw-Hill, 2020.
2. Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
3. Chen, P. P. “The Entity-Relationship Model — Toward a Unified View of Data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.
4. Codd, E. F. “A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
5. Ramakrishnan, R. and Gehrke, J. *Database Management Systems*, 3rd ed., McGraw-Hill, 2003.
6. Gray, J. and Reuter, A. *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1993.
7. Kent, W. “A Simple Guide to Five Normal Forms in Relational Database Theory,” *Communications of the ACM*, vol. 26, no. 2, pp. 120–125, 1983.
8. Oracle Corporation. *MySQL 8.0 Reference Manual — CHECK Constraints, CREATE TRIGGER, SIGNAL, InnoDB Indexes and Transactions*, dev.mysql.com, accessed 2026.
9. Date, C. J. *An Introduction to Database Systems*, 8th ed., Pearson, 2003.
10. MDN Web Docs. “JavaScript Guide,” Mozilla Foundation, developer.mozilla.org, accessed 2026.